Fiche de Révisions Intégrale & Exhaustive : Systèmes d'Exploitation (S7)

Génie Informatique – ENSA Kénitra
Pr. S. Ouatik – Cours Complet, SGF, i-nœuds, FAT, Processus, Shell & TDs

Année Universitaire 2026–2027

Audit de Couverture Totale – Zéro Omission

Cette fiche intègre l'intégralité du corpus théorique, des structures internes (binaires, i-nœuds, PCB), des algorithmes d'ordonnancement, des commandes Linux/Bash avec leurs options et drapeaux, ainsi que les résolutions mathématiques et dimensionnelles des Travaux Dirigés (Série n°1).

1 Chapitre 1 : Généralités des Systèmes d'Exploitation

- **Définition Fondamentale :** Interface logicielle s'intercalant entre le matériel brut (*hardware*) et les applications utilisateurs.
- **Le Syndrome du Disque Non Système :** Erreur critique d'amorçage (*"Disk non system, replace and strike any key"*) survenant lorsque le BIOS ne trouve aucun secteur amorçable (MBR) valide contenant un chargeur de démarrage.
- **Les 2 Missions Cardinales du SE :**
  1. *Machine Virtuelle :* Abstrait la complexité matérielle brute en fournissant des abstractions de haut niveau (fichiers, processus, sockets réseau).
  2. *Gestionnaire de Ressources :* Alloue, arbitre, protège et partage de manière optimale la CPU, la mémoire centrale et les périphériques d'E/S.
- **Typologie des SE :** Monoprogrammés, Multiprogrammés, Temps Partagé (*Time-Sharing*), Temps Réel (*RTOS* – critiques en embarqué et milieu industriel), Distribués.
- **Modèles de Partage des Ressources :**
  - *CPU :* Multiplexage temporel (partage de la puissance de calcul en quanta).
  - *Mémoire Centrale :* Multiplexage spatial (partitionnement, pagination, segmentation).
  - *Ressources Logicielles :* Partage sécurisé des bibliothèques, fichiers et canaux de communication.

2 Chapitre 2 : Système de Gestion de Fichiers (SGF)

- **Modèle Logique :** Suite d'octets (*Byte Stream*) sous UNIX ou enregistrements structurés. Règles de nommage (sensibilité à la casse, extensions indicatives mais non obligatoires pour le noyau, ex. `.c` exigé par `gcc`).
- **Structure d'un Fichier Binaire Exécutable (UNIX) :**
  1. *En-tête (Header) :* Nombre magique (*Magic Number* d'identification), taille du code (*Text size*), taille des données initialisées (*Data size*), taille **BSS** (*Block Started by Symbol* – variables non initialisées allouées à l'exécution sans occuper d'espace disque), table des symboles et point d'entrée (*Entry Point*).
  2. *Segments :* Code machine, Données, bits de relocation (*Relocation bits*) et table des symboles pour le débogage.
- **Appels Système SGF :** `open()`, `read()`, `write()`, `close()`, `creat()`, `unlink()` (suppression de lien/fichier), `lseek()` (déplacement du pointeur de lecture/écriture).
- **Types de Fichiers UNIX :** Ordinaires (`-`), Répertoires (`d`), Fichiers spéciaux mode bloc (`b`), Fichiers spéciaux mode caractère (`c`), Tubes nommés / FIFO (`p`), Liens symboliques (`l`).
- **Liens Physiques vs Symboliques :**
  - *Lien physique (`link`) :* Pointeur direct vers l'i-nœud, même i-nœud, impossible inter-partitions.

  - *Lien symbolique (`symlink`) :* Fichier indépendant contenant le chemin textuel cible, i-nœud distinct, supporte les répertoires et inter-partitions.

- **Allocation Physique (3 Méthodes) :**
  1. *Contiguë :* Blocs consécutifs. Accès direct ultra-rapide mais **fragmentation externe** critique et coût prohibitif de compactage.
  2. *Liste chaînée sur disque :* Chaque bloc contient un pointeur vers le suivant ($\mathcal{O}(n)$ en accès direct).
  3. *Table FAT (File Allocation Table) :* Pointeurs centralisés dans une table en mémoire RAM (valeur -1 = fin de chaîne, 0 = bloc libre).

Méthode de Calcul TD 1 : Localisation dans une FAT & Bitmap vs Liste Chaînée

FAT : On mémorise uniquement le numéro du premier bloc. Chaque entrée donne le bloc suivant jusqu'à -1.
Bitmap vs Liste Chaînée : Soit D bits par adresse disque, B blocs totaux, F blocs libres.
La liste chaînée est plus compacte que le bitmap si : $F < \frac{B}{D}$ (*Ex : pour* $D = 32$ *bits,* $F < \frac{B}{32} \approx 3,125\%$).

Méthode de Calcul TD 2 : Dimensionnement i-nœud UNIX

Capacité bloc d'index : $K = \frac{B}{P}$ (ex : $\frac{1024}{4} = 256$ adresses).
– k Directs ($k \times B$) $\rightarrow$ Simple indirection ($K \times B$) $\rightarrow$ Double indirection ($K^2 \times B$) $\rightarrow$ Triple indirection ($K^3 \times B$).
Fichier 100 000 octets (bloc 1024 o) : 98 blocs de données + 1 bloc d'index simple = **99 blocs physiques** (101 376 octets).

3 Chapitre 3 : Processus & Ordonnancement du Processeur

- **Processus :** Programme en cours d'exécution muni d'un espace d'adressage segmenté (Code, Données, Pile) et d'un **PCB (Process Control Block)**.
- **Modes d'exécution :** Mode Utilisateur (*User mode*) vs Mode Noyau (*Kernel mode*) via interruptions et appels système.
- **Algorithmes CPU :** FCFS (Effet convoyeur), SJF / SRTF (Minimalise temps d'attente), Round Robin (Quantum q), Priorité (Vieillissement contre la famine), Files Multi-Niveaux avec Rétroaction (*Feedback*).

4 Chapitre 6 : Commandes Linux, Redirections & Shell Bash (Intégral)

1. Commandes et Options Essentielles

- `pwd` : Affiche le répertoire courant absolu.
- `ls -l` : Format long (permissions, liens, propriétaire, groupe, taille, date).
- `ls -a` : Affiche tous les fichiers, y compris les cachés (`alinea`).
- `stat <fic>` : Affiche les métadonnées complètes de l'i-nœud.
- `tar -cvzf archive.tar.gz dossier` : Crée (`-c`), affiche les détails (`-v`), compresse avec gzip (`-z`) dans le fichier désigné (`-f`).
- `find <chemin> -name "motif" -type f` : Recherche récursive de fichiers ordinaires (`-type f`). Option `-maxdepth N` pour limiter la profondeur.
- `grep "texte" <fic>` : Recherche de motifs textuels dans les fichiers.
- `ps -l` : État détaillé des processus (UID, PID, PPID, PRI, NI, STAT, TTY).
- `cat <fic>` : Affiche le contenu d'un fichier sur la sortie standard.

2. Redirections et Tubes (`>`, `>>`, `|`)

- `> fichier` : Redirige la sortie standard vers un fichier en mode **écrasement** (crase).
- `>> fichier` : Redirige la sortie standard en mode **ajout** (*append*) à la fin du fichier.
- `cmd1 | cmd2` : Tube (*pipe*) connectant la sortie de `cmd1` directement à l'entrée de `cmd2`.

3. Gestion des Droits d'Accès (`chmod`)

- Classes : Propriétaire (**u**), Groupe (**g**), Autres (**o**), Tous (**a**).
- Mode Octal : Lecture (**r**=4), Écriture (**w**=2), Exécution (**x**=1). *Ex :* `chmod 754 fichier` → $rwx\ r-x\ r--$.
- Mode Symbolique : `chmod u+x,go-w fichier` (ajout/retrait ciblé).

4. Scripts Shell (Bash) – Syntaxe Avancée

- **Shebang :** `#!/bin/bash` obligatoire en première ligne pour désigner l'interpréteur.
- **Variables et Paramètres :** `$0` (nom du script), `$1`, `$2` (arguments positionnels), `$#` (nombre total d'arguments), `$?` (code de retour dernière commande, 0 = succès), `$$` (PID du script).
- **Test conditionnel et opérateurs logiques :**

```
if test $# -ne 2; then
    echo "PARAMETRES MANQUANTS"
else
    if test -r $1 -a -f $1; then # -r (lisible), -f (ordinaire), -a (ET logique)
        cat $1 >> global.sh
    fi
fi
```

- **Boucle de parcours avec `shift` :** `while test $# -ne 0; do ... shift; done` (décale les arguments vers la gauche de 1 cran).

Astuces d'Examen – Pr. S. Ouatik

- **SGF / i-nœuds :** Toujours calculer $K = B/P$ avant de traiter les indirections. Ne pas oublier d'ajouter le bloc d'index dans N_{total}.
- **FAT :** Suivre la chaîne rigoureusement jusqu'à rencontrer -1. Attention aux blocs orphelins.
- **Scripts Shell :** Toujours tester la validité des fichiers (`-r`, `-f`, `-d`) avant toute manipulation.